

Guía Completa de Nginx

Todo lo que necesitas saber sobre Nginx y Virtual Hosts

AlexSilvaa9

15 de marzo de 2026

Índice

1. Introducción a Nginx	5
1.1. Características principales	5
2. Virtual Hosts (vhosts)	5
2.1. ¿Qué es un Virtual Host?	5
2.1.1. Analogía	5
2.2. Tipos de Virtual Hosts	5
2.2.1. Basados en nombre (Name-based)	5
2.2.2. Basados en IP (IP-based)	5
2.2.3. Basados en puerto (Port-based)	6
3. Arquitectura de Nginx	6
3.1. Modelo de procesos	6
3.2. Ventajas del modelo de Nginx	6
3.3. Comparación con Apache	6
4. Estructura de Configuración	6
4.1. Directorios principales	6
4.2. Niveles de contexto	7
5. Ejemplos de Virtual Hosts	7
5.1. Sitio web estático simple	7
5.2. Proxy reverso a aplicación	8
5.3. HTTPS con SSL/TLS	8
5.4. Múltiples aplicaciones en subdominios	9
6. La directiva try_files	9
6.1. ¿Qué es try_files?	9
6.2. Sintaxis	10
6.3. Funcionamiento paso a paso	10
6.4. Casos de uso	10
6.4.1. SPA (React, Vue, Angular)	10
6.4.2. Sitio estático	10
6.4.3. WordPress/PHP	10
7. Directivas Importantes	10
7.1. Listen	10
7.2. Server Name	10
7.3. Root e Index	11
7.4. Location - Prioridades	11
8. DNS y Virtual Hosts	11
8.1. Flujo completo	11
8.1.1. Paso 1: Configuración DNS	11
8.1.2. Paso 2: El navegador hace la petición	12
8.1.3. Paso 3: Nginx elige el virtual host	12

9. Habilitar y Deshabilitar Sitios	12
9.1. Concepto	12
9.2. Proceso completo	12
9.2.1. 1. Crear configuración	12
9.2.2. 2. Habilitar el sitio (crear symlink)	12
9.2.3. 3. Verificar y recargar	12
9.2.4. 4. Deshabilitar sitio	12
10.Proxy Reverso	13
10.1. Headers esenciales	13
10.2. Upstream y Load Balancing	13
10.2.1. Round-robin (por defecto)	13
10.2.2. Least connections	13
10.2.3. IP Hash	14
10.2.4. Con pesos y parámetros	14
11.Caché en Nginx	14
11.1. Caché de Proxy	14
11.2. Caché de archivos estáticos	14
12.Seguridad	15
12.1. Rate Limiting	15
12.2. Limit de conexiones	15
12.3. Control de acceso	15
12.4. Headers de seguridad	15
12.5. Autenticación básica	15
13.Redirects y Rewrites	16
13.1. Return (más eficiente)	16
13.2. Rewrite	16
14.SSL/TLS	16
14.1. Configuración SSL completa	16
14.2. Let's Encrypt	17
15.Logs y Monitoring	17
15.1. Formatos de log personalizados	17
15.2. Analizar logs	18
16.Variables de Nginx	18
16.1. Variables principales	18
17.Performance Tuning	18
18.Troubleshooting	20
18.1. Comandos de diagnóstico	20
18.2. Errores comunes y soluciones	20
18.2.1. Could not build server_names_hash	20
18.2.2. 413 Request Entity Too Large	20
18.2.3. 502 Bad Gateway	20
18.2.4. Permission denied	21

19. Comandos Esenciales	21
19.1. Gestión del servicio	21
19.2. Comandos de Nginx	21
20. Casos de Uso Avanzados	21
20.1. Microservicios	21
20.2. SPA con API backend	22
20.3. Multi-tenant (SaaS)	22
21. Snippets Reutilizables	23
21.1. SSL Parameters	23
21.2. Security Headers	23
21.3. Static Cache	23
21.4. Uso de snippets	23
22. Ejemplo Práctico Completo	24
22.1. Escenario	24
22.2. Paso 1: Configuración DNS	24
22.3. Paso 2: Crear directorios	24
22.4. Paso 3: Configuración del sitio principal	24
22.5. Paso 4: Configuración del blog	24
22.6. Paso 5: Configuración de la API	25
22.7. Paso 6: Habilitar y verificar	25
23. Conclusión	25
23.1. Puntos clave	25
23.2. Recursos adicionales	26

1. Introducción a Nginx

Nginx (pronunciado .^{en}engine-x”) es un servidor web de código abierto, conocido por su alto rendimiento, estabilidad, configuración simple y bajo consumo de recursos.

1.1. Características principales

- Servidor web HTTP de alto rendimiento
- Proxy reverso para HTTP, HTTPS, SMTP, POP3 e IMAP
- Balanceador de carga
- Caché HTTP
- Manejo eficiente de archivos estáticos
- Soporte para FastCGI, uWSGI, SCGI

2. Virtual Hosts (vhosts)

2.1. ¿Qué es un Virtual Host?

Un **virtual host** es una configuración que permite que un **mismo servidor web** (una sola máquina con una IP) sirva **múltiples sitios web o aplicaciones diferentes**.

2.1.1. Analogía

Imagina un edificio de apartamentos (el servidor):

- **Una dirección** (una IP)
- **Múltiples apartamentos** (diferentes sitios web)
- **El portero** (Nginx) dirige a cada visitante al apartamento correcto según el nombre que pida

2.2. Tipos de Virtual Hosts

2.2.1. Basados en nombre (Name-based)

El más común. Usa el **nombre de dominio** para distinguir sitios:

- `ejemplo1.com` → Sitio A
- `ejemplo2.com` → Sitio B
- **Misma IP**, diferentes dominios

2.2.2. Basados en IP (IP-based)

Cada sitio tiene su propia IP:

- `192.168.1.10` → Sitio A
- `192.168.1.11` → Sitio B

2.2.3. Basados en puerto (Port-based)

Mismo dominio/IP, diferentes puertos:

- `ejemplo.com:80` → Sitio A
- `ejemplo.com:8080` → Sitio B

3. Arquitectura de Nginx

3.1. Modelo de procesos

Nginx usa un modelo **asíncrono y basado en eventos** (event-driven):

Estructura de procesos

Master Process (root, gestiona workers)

Worker Process Worker Process Worker Process

(atiende miles de conexiones cada uno)

3.2. Ventajas del modelo de Nginx

- Un worker puede manejar miles de conexiones simultáneas
- Muy eficiente en memoria
- No bloquea operaciones (non-blocking I/O)

3.3. Comparación con Apache

- **Apache:** Crea un proceso/thread por conexión (más pesado)
- **Nginx:** Event-driven, maneja múltiples conexiones por worker

4. Estructura de Configuración

4.1. Directorios principales

```
1 /etc/nginx/
2     nginx.conf           # Configuración principal
3     sites-available/     # Configuraciones disponibles
4         sitio1.conf
5         sitio2.conf
6         default
7     sites-enabled/       # Symlinks a sitios activos
8         sitio1.conf -> ../sites-available/sitio1.conf
```

```
9         default -> ../sites-available/default
10     conf.d/                # Configuraciones adicionales
11     snippets/              # Fragmentos reutilizables
12     mime.types              # Tipos MIME
```

Listing 1: Estructura de directorios de Nginx

4.2. Niveles de contexto

```
1 # CONTEXTO MAIN (global)
2 user www-data;
3 worker_processes auto;
4 pid /run/nginx.pid;
5
6 events {
7     # CONTEXTO EVENTS
8     worker_connections 1024;
9     use epoll;
10 }
11
12 http {
13     # CONTEXTO HTTP
14     include /etc/nginx/mime.types;
15     default_type application/octet-stream;
16
17     sendfile on;
18     keepalive_timeout 65;
19
20     access_log /var/log/nginx/access.log;
21     error_log /var/log/nginx/error.log;
22
23     upstream backend {
24         server localhost:3000;
25         server localhost:3001;
26     }
27
28     server {
29         # CONTEXTO SERVER (virtual host)
30         listen 80;
31         server_name ejemplo.com;
32
33         location / {
34             # CONTEXTO LOCATION
35             root /var/www/html;
36             index index.html;
37         }
38     }
39
40     include /etc/nginx/conf.d/*.conf;
41     include /etc/nginx/sites-enabled/*;
42 }
```

Listing 2: Estructura completa de nginx.conf

5. Ejemplos de Virtual Hosts

5.1. Sitio web estático simple

```
1 server {
2     listen 80;
3     listen [::]:80;
4
5     server_name miapp.com www.miapp.com;
6
7     root /var/www/miapp;
8     index index.html index.htm;
9
10    access_log /var/log/nginx/miapp-access.log;
11    error_log /var/log/nginx/miapp-error.log;
12
13    location / {
14        try_files $uri $uri/ =404;
15    }
16 }
```

Listing 3: sites-available/miapp.com.conf

5.2. Proxy reverso a aplicación

```
1 server {
2     listen 80;
3     server_name api.miapp.com;
4
5     location / {
6         proxy_pass http://localhost:3000;
7         proxy_http_version 1.1;
8
9         proxy_set_header Upgrade $http_upgrade;
10        proxy_set_header Connection 'upgrade';
11        proxy_set_header Host $host;
12        proxy_set_header X-Real-IP $remote_addr;
13        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
14        proxy_set_header X-Forwarded-Proto $scheme;
15
16        proxy_cache_bypass $http_upgrade;
17    }
18 }
```

Listing 4: Proxy a Node.js/Python/etc

5.3. HTTPS con SSL/TLS

```
1 server {
2     listen 80;
3     server_name miapp.com www.miapp.com;
4     return 301 https://$server_name$request_uri;
5 }
6
7 server {
8     listen 443 ssl http2;
9     listen [::]:443 ssl http2;
10
11    server_name miapp.com www.miapp.com;
12
13    ssl_certificate /etc/ssl/certs/miapp.crt;
```

```
14     ssl_certificate_key /etc/ssl/private/miapp.key;
15
16     ssl_protocols TLSv1.2 TLSv1.3;
17     ssl_ciphers HIGH:!aNULL:!MD5;
18     ssl_prefer_server_ciphers on;
19
20     root /var/www/miapp;
21     index index.html;
22
23     location / {
24         try_files $uri $uri/ =404;
25     }
26 }
```

Listing 5: Configuración SSL completa

5.4. Múltiples aplicaciones en subdominios

```
1 # Aplicación principal
2 server {
3     listen 80;
4     server_name miempresa.com www.miempresa.com;
5     root /var/www/principal;
6     index index.html;
7 }
8
9 # Blog
10 server {
11     listen 80;
12     server_name blog.miempresa.com;
13
14     location / {
15         proxy_pass http://localhost:2368;
16         proxy_set_header Host $host;
17     }
18 }
19
20 # API
21 server {
22     listen 80;
23     server_name api.miempresa.com;
24
25     location / {
26         proxy_pass http://localhost:3000;
27         proxy_set_header Host $host;
28     }
29 }
```

Listing 6: Multi-sitio con subdominios

6. La directiva try_files

6.1. ¿Qué es try_files?

`try_files` es una directiva que **intenta servir archivos en un orden específico** hasta encontrar uno que exista.

6.2. Sintaxis

```
1 try_files archivo1 archivo2 ... fallback;
```

6.3. Funcionamiento paso a paso

Cuando llega una petición `http://miapp.com/about`:

1. **Paso 1:** Nginx busca `$uri` → `/var/www/miapp/about` (archivo)
2. **Paso 2:** Nginx busca `$uri/` → `/var/www/miapp/about/` (directorio)
3. **Paso 3:** Nginx sirve el fallback → `/index.html`

6.4. Casos de uso

6.4.1. SPA (React, Vue, Angular)

```
1 location / {  
2     try_files $uri $uri/ /index.html;  
3 }
```

6.4.2. Sitio estático

```
1 location / {  
2     try_files $uri $uri/ =404;  
3 }
```

6.4.3. WordPress/PHP

```
1 location / {  
2     try_files $uri $uri/ /index.php?$args;  
3 }
```

7. Directivas Importantes

7.1. Listen

```
1 listen 80;                # IPv4, puerto 80  
2 listen [::]:80;           # IPv6, puerto 80  
3 listen 443 ssl;           # HTTPS  
4 listen 8080;              # Puerto personalizado
```

7.2. Server Name

```
1 server_name ejemplo.com;  
2 server_name ejemplo.com www.ejemplo.com;  
3 server_name *.ejemplo.com;  
4 server_name ~^(?<subdomain>.+)\.ejemplo\.com$;
```

7.3. Root e Index

```
1 root /var/www/html;
2 index index.html index.php;
```

7.4. Location - Prioridades

Las directivas `location` se evalúan en este orden:

1. `location = /exacto` - Match exacto (PRIORIDAD MÁXIMA)
2. `location ^~ /img/` - Preferential prefix
3. `location ~ php$` - Regex case-sensitive
4. `location ~* (jpg|png)$` - Regex case-insensitive
5. `location /` - Prefix match (PRIORIDAD MÍNIMA)

```
1 server {
2     listen 80;
3     server_name test.local;
4
5     location = / {
6         return 200 "Exacto: raiz\n";
7     }
8
9     location / {
10        return 200 "Prefix: cualquier cosa\n";
11    }
12
13    location /imagenes/ {
14        return 200 "Prefix: imagenes\n";
15    }
16
17    location ~ /\.png$ {
18        return 200 "Regex: archivos PNG\n";
19    }
20 }
```

Listing 7: Ejemplo de prioridades

8. DNS y Virtual Hosts

8.1. Flujo completo

8.1.1. Paso 1: Configuración DNS

En tu proveedor DNS (Cloudflare, GoDaddy, Route53):

Registros DNS			
Type	Name	Value	TTL
A	miapp.com	203.0.113.10	3600
A	blog.com	203.0.113.10	3600
A	tienda.com	203.0.113.10	3600

Múltiples dominios apuntan a la MISMA IP

8.1.2. Paso 2: El navegador hace la petición

```
1 GET / HTTP/1.1
2 Host: miapp.com
```

Clave: El navegador envía el nombre del dominio en la cabecera `Host:`

8.1.3. Paso 3: Nginx elige el virtual host

Nginx compara la cabecera `Host:` con `server_name`:

- Si `Host: miapp.com` → sirve `/var/www/miapp`
- Si `Host: blog.com` → sirve `/var/www/blog`
- Si `Host: tienda.com` → sirve `/var/www/tienda`

9. Habilitar y Deshabilitar Sitios

9.1. Concepto

Nginx usa dos directorios:

- `sites-available/` - Todas las configuraciones (activas o no)
- `sites-enabled/` - Solo las configuraciones ACTIVAS

9.2. Proceso completo

9.2.1. 1. Crear configuración

```
1 sudo nano /etc/nginx/sites-available/miapp.conf
```

9.2.2. 2. Habilitar el sitio (crear symlink)

```
1 sudo ln -s /etc/nginx/sites-available/miapp.conf \
2      /etc/nginx/sites-enabled/
```

9.2.3. 3. Verificar y recargar

```
1 sudo nginx -t
2 sudo nginx -s reload
```

9.2.4. 4. Deshabilitar sitio

```
1 sudo rm /etc/nginx/sites-enabled/miapp.conf
2 sudo nginx -s reload
```

Importante: Solo se borra el symlink, la configuración permanece en `sites-available/`

10. Proxy Reverso

10.1. Headers esenciales

```
1 location / {
2     proxy_pass http://backend;
3
4     # Headers esenciales
5     proxy_set_header Host $host;
6     proxy_set_header X-Real-IP $remote_addr;
7     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
8     proxy_set_header X-Forwarded-Proto $scheme;
9     proxy_set_header X-Forwarded-Host $host;
10    proxy_set_header X-Forwarded-Port $server_port;
11
12    # WebSockets
13    proxy_http_version 1.1;
14    proxy_set_header Upgrade $http_upgrade;
15    proxy_set_header Connection "upgrade";
16
17    # Timeouts
18    proxy_connect_timeout 60s;
19    proxy_send_timeout 60s;
20    proxy_read_timeout 60s;
21
22    # Buffers
23    proxy_buffering on;
24    proxy_buffer_size 4k;
25    proxy_buffers 8 4k;
26
27    proxy_cache_bypass $http_upgrade;
28 }
```

Listing 8: Configuración completa de proxy

10.2. Upstream y Load Balancing

10.2.1. Round-robin (por defecto)

```
1 upstream backend {
2     server backend1.com;
3     server backend2.com;
4     server backend3.com;
5 }
```

10.2.2. Least connections

```
1 upstream backend {
2     least_conn;
3     server backend1.com;
4     server backend2.com;
5 }
```

10.2.3. IP Hash

```
1 upstream backend {
2     ip_hash;
3     server backend1.com;
4     server backend2.com;
5 }
```

10.2.4. Con pesos y parámetros

```
1 upstream backend {
2     server backend1.com weight=3;
3     server backend2.com max_fails=3 fail_timeout=30s;
4     server backend3.com backup;
5     server backend4.com down;
6 }
```

11. Caché en Nginx

11.1. Caché de Proxy

```
1 http {
2     proxy_cache_path /var/cache/nginx
3                     levels=1:2
4                     keys_zone=my_cache:10m
5                     max_size=1g
6                     inactive=60m
7                     use_temp_path=off;
8
9     server {
10         location / {
11             proxy_pass http://backend;
12             proxy_cache my_cache;
13             proxy_cache_valid 200 10m;
14             proxy_cache_valid 404 1m;
15
16             add_header X-Cache-Status $upstream_cache_status;
17
18             proxy_cache_key "$scheme$request_method$host$request_uri";
19             proxy_cache_bypass $cookie_nocache $arg_nocache;
20         }
21     }
22 }
```

Listing 9: Configuración de caché

11.2. Caché de archivos estáticos

```
1 location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
2     expires 1y;
3     add_header Cache-Control "public, immutable";
4     access_log off;
5 }
```

12. Seguridad

12.1. Rate Limiting

```
1 http {
2     limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;
3
4     server {
5         location /api/ {
6             limit_req zone=mylimit burst=20 nodelay;
7         }
8     }
9 }
```

Listing 10: Protección contra DDoS

12.2. Limit de conexiones

```
1 http {
2     limit_conn_zone $binary_remote_addr zone=addr:10m;
3
4     server {
5         location /download/ {
6             limit_conn addr 2;
7         }
8     }
9 }
```

12.3. Control de acceso

```
1 location /admin {
2     allow 192.168.1.0/24;
3     allow 10.0.0.1;
4     deny all;
5 }
```

12.4. Headers de seguridad

```
1 add_header X-Frame-Options "SAMEORIGIN" always;
2 add_header X-Content-Type-Options "nosniff" always;
3 add_header X-XSS-Protection "1; mode=block" always;
4 add_header Referrer-Policy "no-referrer-when-downgrade" always;
5 add_header Content-Security-Policy
6     "default-src 'self' http: https: data: blob: 'unsafe-inline'" always;
7
8 server_tokens off;
```

12.5. Autenticación básica

```
1 # Crear archivo de contraseñas
2 sudo htpasswd -c /etc/nginx/.htpasswd usuario1
```

```
1 location /admin {
2     auth_basic "Area restringida";
3     auth_basic_user_file /etc/nginx/.htpasswd;
4 }
```

13. Redirects y Rewrites

13.1. Return (más eficiente)

```
1 # Redirect permanente
2 server {
3     listen 80;
4     server_name old-domain.com;
5     return 301 https://new-domain.com$request_uri;
6 }
7
8 # HTTP a HTTPS
9 server {
10    listen 80;
11    return 301 https://$server_name$request_uri;
12 }
13
14 # Redirect de página
15 location /old-page {
16     return 301 /new-page;
17 }
```

13.2. Rewrite

```
1 # Rewrite simple
2 rewrite ^/old/(.*)$ /new/$1 permanent;
3
4 # Con condiciones
5 if ($host != 'www.ejemplo.com') {
6     rewrite ^/(.*)$ https://www.ejemplo.com/$1 permanent;
7 }
```

Flags de rewrite:

- last - Detiene y busca nueva location
- break - Detiene en esta location
- redirect - Redirect temporal (302)
- permanent - Redirect permanente (301)

14. SSL/TLS

14.1. Configuración SSL completa

```
1 server {
2     listen 443 ssl http2;
3     listen [::]:443 ssl http2;
4     server_name ejemplo.com;
5
6     # Certificados
7     ssl_certificate /etc/letsencrypt/live/ejemplo.com/fullchain.pem;
8     ssl_certificate_key /etc/letsencrypt/live/ejemplo.com/privkey.pem;
9
10    # Protocolos modernos
11    ssl_protocols TLSv1.2 TLSv1.3;
12
13    # Cifrados seguros
14    ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
15    ssl_prefer_server_ciphers off;
16
17    # OCSP Stapling
18    ssl_stapling on;
19    ssl_stapling_verify on;
20    resolver 8.8.8.8 8.8.4.4 valid=300s;
21
22    # Sesiones SSL
23    ssl_session_cache shared:SSL:10m;
24    ssl_session_timeout 10m;
25    ssl_session_tickets off;
26
27    # HSTS
28    add_header Strict-Transport-Security
29        "max-age=31536000; includeSubDomains; preload" always;
30
31    # Diffie-Hellman
32    ssl_dhparam /etc/nginx/dhparam.pem;
33 }
```

Listing 11: SSL/TLS moderno y seguro

14.2. Let's Encrypt

```
1 # Instalar certbot
2 sudo apt install certbot python3-certbot-nginx
3
4 # Obtener certificado
5 sudo certbot --nginx -d ejemplo.com -d www.ejemplo.com
6
7 # Auto-renovaci n
8 sudo certbot renew --dry-run
```

15. Logs y Monitoring

15.1. Formatos de log personalizados

```
1 http {
2     log_format main '$remote_addr - $remote_user [$time_local] '
3                     '"$request" $status $body_bytes_sent '
4                     '"$http_referer" "$http_user_agent" '
5                     '$request_time';
```

```
6
7     log_format json escape=json '{'
8         '"time":'$time_local','
9         '"remote_addr":'$remote_addr','
10        '"request":'$request','
11        '"status":'$status','
12        '"body_bytes_sent":'$body_bytes_sent','
13        '"request_time":'$request_time'
14    '}'
15
16    access_log /var/log/nginx/access.log main;
17
18    server {
19        access_log /var/log/nginx/miapp-access.log json;
20        error_log /var/log/nginx/miapp-error.log warn;
21    }
22 }
```

15.2. Analizar logs

```
1 # Ver ltimas peticiones
2 tail -f /var/log/nginx/access.log
3
4 # IPs con m s peticiones
5 awk '{print $1}' /var/log/nginx/access.log | \
6     sort | uniq -c | sort -rn | head -10
7
8 # C digos de estado
9 awk '{print $9}' /var/log/nginx/access.log | \
10    sort | uniq -c | sort -rn
11
12 # URLs m s visitadas
13 awk '{print $7}' /var/log/nginx/access.log | \
14    sort | uniq -c | sort -rn | head -10
```

16. Variables de Nginx

16.1. Variables principales

17. Performance Tuning

```
1 user www-data;
2 worker_processes auto;
3 worker_rlimit_nofile 65535;
4
5 events {
6     worker_connections 4096;
7     use epoll;
8     multi_accept on;
9 }
10
11 http {
12     # Sendfile y TCP
13     sendfile on;
14     tcp_nopush on;
15     tcp_nodelay on;
```

Variable	Descripción
\$host	Nombre del host (ejemplo.com)
\$uri	URI de la petición (/path/to/file)
\$request_uri	URI completa con query string
\$args	Query string (?id=123&name=test)
\$remote_addr	IP del cliente
\$server_addr	IP del servidor
\$server_port	Puerto del servidor
\$scheme	http o https
\$request_method	GET, POST, PUT, etc.
\$http_user_agent	User agent del navegador
\$http_referer	Página de referencia
\$status	Código de estado HTTP
\$body_bytes_sent	Bytes enviados (sin headers)
\$request_time	Tiempo total de procesamiento

Cuadro 1: Variables principales de Nginx

```
16
17 # Timeouts
18 keepalive_timeout 30;
19 keepalive_requests 100;
20 reset_timedout_connection on;
21 client_body_timeout 10;
22 send_timeout 10;
23
24 # Buffers
25 client_body_buffer_size 128k;
26 client_max_body_size 10m;
27 client_header_buffer_size 1k;
28 large_client_header_buffers 4 4k;
29
30 # Gzip
31 gzip on;
32 gzip_vary on;
33 gzip_comp_level 5;
34 gzip_min_length 256;
35 gzip_types
36     text/plain
37     text/css
38     text/xml
39     text/javascript
40     application/json
41     application/javascript
42     application/xml+rss;
43
44 # File cache
45 open_file_cache max=200000 inactive=20s;
46 open_file_cache_valid 30s;
47 open_file_cache_min_uses 2;
48 open_file_cache_errors on;
49 }
```

Listing 12: Optimización de rendimiento

18. Troubleshooting

18.1. Comandos de diagnóstico

```
1 # Verificar sintaxis
2 sudo nginx -t
3
4 # Ver configuraci n completa
5 sudo nginx -T
6
7 # Ver versi n y m dulos
8 nginx -V
9
10 # Ver procesos
11 ps aux | grep nginx
12
13 # Ver puertos
14 netstat -tlnp | grep nginx
15
16 # Ver logs en tiempo real
17 tail -f /var/log/nginx/error.log
```

18.2. Errores comunes y soluciones

18.2.1. Could not build server_names_hash

```
1 http {
2     server_names_hash_bucket_size 64;
3 }
```

18.2.2. 413 Request Entity Too Large

```
1 http {
2     client_max_body_size 100M;
3 }
```

18.2.3. 502 Bad Gateway

- Backend no está corriendo
- Firewall bloqueando conexión
- Socket PHP-FPM incorrecto

```
1 # Verificar backend
2 curl http://localhost:3000
3
4 # Ver logs
5 tail -f /var/log/nginx/error.log
```

18.2.4. Permission denied

```
1 # Verificar permisos
2 ls -la /var/www/miapp
3
4 # Cambiar due o
5 sudo chown -R www-data:www-data /var/www/miapp
```

19. Comandos Esenciales

19.1. Gestión del servicio

```
1 # Iniciar
2 sudo systemctl start nginx
3
4 # Detener
5 sudo systemctl stop nginx
6
7 # Reiniciar
8 sudo systemctl restart nginx
9
10 # Recargar (sin downtime)
11 sudo systemctl reload nginx
12
13 # Ver estado
14 sudo systemctl status nginx
15
16 # Auto-start en boot
17 sudo systemctl enable nginx
```

19.2. Comandos de Nginx

```
1 # Test de configuraci n
2 sudo nginx -t
3
4 # Dump de configuraci n
5 sudo nginx -T
6
7 # Reload
8 sudo nginx -s reload
9
10 # Stop
11 sudo nginx -s stop
12
13 # Graceful shutdown
14 sudo nginx -s quit
15
16 # Reabrir logs
17 sudo nginx -s reopen
```

20. Casos de Uso Avanzados

20.1. Microservicios

```
1 server {
2     listen 80;
3     server_name api.ejemplo.com;
4
5     location /users {
6         proxy_pass http://localhost:3001;
7     }
8
9     location /products {
10        proxy_pass http://localhost:3002;
11    }
12
13    location /payments {
14        proxy_pass http://localhost:3003;
15    }
16 }
```

20.2. SPA con API backend

```
1 server {
2     listen 80;
3     server_name app.ejemplo.com;
4     root /var/www/frontend/dist;
5
6     # Frontend
7     location / {
8         try_files $uri $uri/ /index.html;
9     }
10
11    # API
12    location /api {
13        proxy_pass http://localhost:3000;
14        proxy_set_header Host $host;
15    }
16
17    # WebSocket
18    location /ws {
19        proxy_pass http://localhost:3000;
20        proxy_http_version 1.1;
21        proxy_set_header Upgrade $http_upgrade;
22        proxy_set_header Connection "Upgrade";
23    }
24
25    # Archivos est ticos
26    location /static {
27        expires 1y;
28        add_header Cache-Control "public, immutable";
29    }
30 }
```

20.3. Multi-tenant (SaaS)

```
1 map $http_host $tenant {
2     cliente1.miapp.com "cliente1";
3     cliente2.miapp.com "cliente2";
4     default "demo";
5 }
```

```
6
7 server {
8     listen 80;
9     server_name *.miapp.com;
10
11     location / {
12         proxy_pass http://localhost:3000;
13         proxy_set_header X-Tenant-ID $tenant;
14         proxy_set_header Host $host;
15     }
16 }
```

21. Snippets Reutilizables

21.1. SSL Parameters

```
1 ssl_protocols TLSv1.2 TLSv1.3;
2 ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
3 ssl_prefer_server_ciphers off;
4 ssl_session_cache shared:SSL:10m;
5 add_header Strict-Transport-Security "max-age=31536000" always;
```

Listing 13: /etc/nginx/snippets/ssl-params.conf

21.2. Security Headers

```
1 add_header X-Frame-Options "SAMEORIGIN" always;
2 add_header X-Content-Type-Options "nosniff" always;
3 add_header X-XSS-Protection "1; mode=block" always;
4 server_tokens off;
```

Listing 14: /etc/nginx/snippets/security-headers.conf

21.3. Static Cache

```
1 location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
2     expires 1y;
3     add_header Cache-Control "public, immutable";
4 }
```

Listing 15: /etc/nginx/snippets/static-cache.conf

21.4. Uso de snippets

```
1 server {
2     listen 443 ssl http2;
3
4     include snippets/ssl-params.conf;
5     include snippets/security-headers.conf;
6     include snippets/static-cache.conf;
7 }
```

22. Ejemplo Práctico Completo

22.1. Escenario

Servidor en 203.0.113.50 con:

- `empresa.com` → Sitio corporativo
- `blog.empresa.com` → Blog
- `api.empresa.com` → API backend

22.2. Paso 1: Configuración DNS

Registros DNS

Type	Name	Value	TTL
A	empresa.com	203.0.113.50	3600
A	www.empresa.com	203.0.113.50	3600
A	blog.empresa.com	203.0.113.50	3600
A	api.empresa.com	203.0.113.50	3600

22.3. Paso 2: Crear directorios

```
1 sudo mkdir -p /var/www/empresa
2 sudo mkdir -p /var/www/blog
```

22.4. Paso 3: Configuración del sitio principal

```
1 server {
2     listen 80;
3     server_name empresa.com www.empresa.com;
4
5     root /var/www/empresa;
6     index index.html;
7
8     location / {
9         try_files $uri $uri/ /index.html;
10    }
11 }
```

Listing 16: `/etc/nginx/sites-available/empresa.conf`

22.5. Paso 4: Configuración del blog

```
1 server {
2     listen 80;
3     server_name blog.empresa.com;
4
5     root /var/www/blog;
6     index index.html;
7
8     location / {
9         try_files $uri $uri/ =404;
```

```
10 }  
11 }
```

Listing 17: /etc/nginx/sites-available/blog.conf

22.6. Paso 5: Configuración de la API

```
1 server {  
2     listen 80;  
3     server_name api.empresa.com;  
4  
5     location / {  
6         proxy_pass http://localhost:3000;  
7         proxy_set_header Host $host;  
8         proxy_set_header X-Real-IP $remote_addr;  
9     }  
10 }
```

Listing 18: /etc/nginx/sites-available/api.conf

22.7. Paso 6: Habilitar y verificar

```
1 # Habilitar sitios  
2 sudo ln -s /etc/nginx/sites-available/empresa.conf \  
3     /etc/nginx/sites-enabled/  
4 sudo ln -s /etc/nginx/sites-available/blog.conf \  
5     /etc/nginx/sites-enabled/  
6 sudo ln -s /etc/nginx/sites-available/api.conf \  
7     /etc/nginx/sites-enabled/  
8  
9 # Verificar configuraci n  
10 sudo nginx -t  
11  
12 # Recargar Nginx  
13 sudo nginx -s reload  
14  
15 # Probar  
16 curl http://empresa.com  
17 curl http://blog.empresa.com  
18 curl http://api.empresa.com
```

23. Conclusión

Esta guía cubre los aspectos fundamentales y avanzados de Nginx, desde la configuración básica de virtual hosts hasta optimización de rendimiento y casos de uso complejos.

23.1. Puntos clave

1. Nginx es un servidor web eficiente y versátil
2. Los virtual hosts permiten múltiples sitios en un servidor
3. La configuración es modular y reutilizable
4. La seguridad debe ser una prioridad
5. El monitoreo y los logs son esenciales

23.2. Recursos adicionales

- Documentación oficial: <https://nginx.org/en/docs/>
- Nginx Admin Guide: <https://docs.nginx.com/nginx/admin-guide/>
- DigitalOcean Tutorials: <https://www.digitalocean.com/community/tags/nginx>